

Docker Learning Lab

Applicazione Flask + PostgreSQL + pgAdmin con Docker Compose

Obiettivo del laboratorio. Creare un piccolo ambiente applicativo realistico per capire il funzionamento di Docker Desktop, delle immagini, dei container, di Docker Compose, della rete tra servizi, delle variabili d'ambiente e dei volumi persistenti.

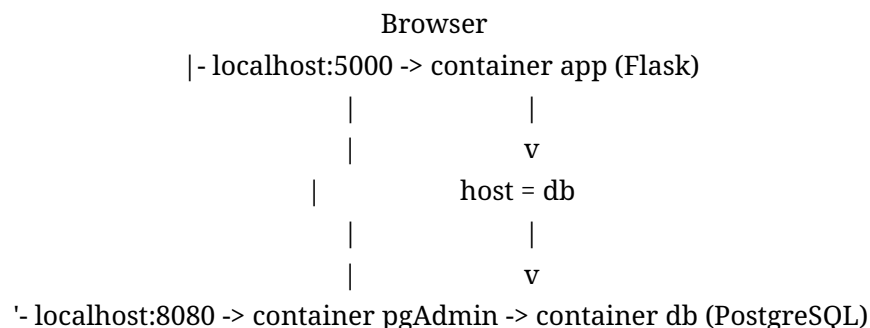
1. Idea del progetto

Il progetto è una piccola applicazione web in Python/Flask collegata a un database PostgreSQL. L'utente può inserire un nome e verificare quante persone con lo stesso nome sono presenti nel database. I dati non vengono mostrati direttamente nella pagina principale, ma possono essere consultati e gestiti tramite pgAdmin.

Servizio	Ruolo	Accesso
app	Applicazione Flask	http://localhost:5000
db	Database PostgreSQL	Rete interna Docker, porta 5432
pgadmin	Interfaccia grafica PostgreSQL	http://localhost:8080

2. Architettura Docker

Docker Compose descrive l'intero ambiente. Ogni componente gira in un container separato, ma i container appartengono alla stessa rete Docker e possono comunicare usando il nome del servizio.



3. Struttura dei file

- **app/app.py:** Contiene la logica Flask e le query SQL verso PostgreSQL.
- **requirements.txt:** Elenca le dipendenze Python necessarie all'applicazione.
- **Dockerfile:** Descrive come costruire l'immagine personalizzata dell'applicazione.
- **docker-compose.yml:** Definisce i tre servizi, le porte, le variabili d'ambiente, la rete e i volumi.
- **.env:** Contiene le variabili locali usate dai container, incluse le credenziali di sviluppo.
- **.gitignore:** Evita che file locali o sensibili vengano inclusi accidentalmente nel versionamento.

- **README.md:** Riassume scopo, stack e modalità di esecuzione del laboratorio.

4. Il Dockerfile

FROM python:3.13-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app/ .

CMD ["python", "app.py"]

Il Dockerfile parte da un'immagine Python ufficiale, definisce una directory di lavoro, copia e installa le dipendenze e infine copia il codice dell'applicazione. Il comando CMD stabilisce cosa deve essere eseguito quando il container viene avviato.

5. Docker Compose

Docker Compose permette di avviare e gestire più servizi con un unico file. Nel laboratorio abbiamo definito i servizi app, db e pgadmin. Il servizio app comunica con PostgreSQL usando DB_HOST=db. Non si usa localhost perché, all'interno di un container, localhost identifica il container stesso.

Comando principale: docker compose up --build

L'opzione --build ricostruisce l'immagine dell'applicazione quando il codice o il Dockerfile cambiano. Il comando docker compose down ferma e rimuove i container dello stack, ma mantiene i volumi.

6. Persistenza dei dati

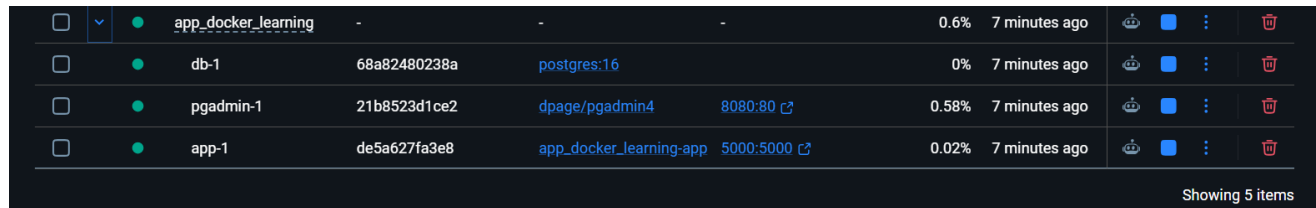
I container possono essere fermati e ricreati. Per evitare che i dati importanti vadano persi, abbiamo usato volumi Docker separati dal ciclo di vita dei container.

- **postgres_data:** mantiene i dati del database PostgreSQL.
- **pgadmin_data:** mantiene la configurazione di pgAdmin e i server registrati.

Attenzione: docker compose down -v rimuove anche i volumi. In un laboratorio può essere utile per ripartire da zero, ma comporta la perdita dei dati persistenti.

7. Docker Desktop

Docker Desktop è stato usato come interfaccia grafica per osservare e gestire l'ambiente. La sezione Containers mostra lo stack `app_docker_learning` e i singoli container che lo compongono. Per ogni container è possibile vedere l'immagine utilizzata, lo stato, le porte esposte, il consumo di risorse e le azioni di start, stop e rimozione.



☐	●	app_docker_learning	-	-	-	0.6%	7 minutes ago				
☐	●	db-1	68a82480238a	postgres:16		0%	7 minutes ago				
☐	●	pgadmin-1	21b8523d1ce2	dpage/pgadmin4	8080:80	0.58%	7 minutes ago				
☐	●	app-1	de5a627fa3e8	app_docker_learning-app	5000:5000	0.02%	7 minutes ago				

Showing 5 items

Figura 1 - I container del laboratorio visualizzati in Docker Desktop.

Nello stack si distinguono `db-1` basato su `postgres:16`, `pgadmin-1` basato su `dpage/pgadmin4` e `app-1` basato sull'immagine personalizzata dell'applicazione. Le porte `5000:5000` e `8080:80` mostrano il mapping tra il PC Windows e i servizi eseguiti nei container.

8. pgAdmin dentro Docker

pgAdmin è stato eseguito anch'esso come container. Dal browser si accede all'interfaccia tramite `localhost:8080`. Per collegarsi al database PostgreSQL viene usato come host il nome del servizio Docker, cioè `db`.

Da pgAdmin è possibile esplorare il database tramite l'albero Databases -> `dockerdb` -> Schemas -> `public` -> Tables -> `persone`, oppure usare il Query Tool per eseguire query SQL, ad esempio `SELECT * FROM persone`.

9. Variabili d'ambiente e credenziali

Le credenziali di sviluppo sono state spostate dal `docker-compose.yml` a un file `.env`. Il file Compose usa riferimenti come `${POSTGRES_PASSWORD}`. Questo separa configurazione e codice e permette di cambiare facilmente le impostazioni tra ambienti diversi.

10. Concetti Docker appresi

- Differenza tra immagine e container.
- Download di immagini già pronte da Docker Hub.
- Build di un'immagine personalizzata tramite Dockerfile.
- Esecuzione coordinata di più servizi con Docker Compose.
- Comunicazione tra container tramite rete Docker e nome del servizio.
- Port mapping tra host Windows e container.
- Persistenza dei dati mediante volumi.
- Separazione tra applicazione, database e strumenti di amministrazione.

- Ricostruzione dell'immagine dopo modifiche al codice.
- Uso di variabili d'ambiente tramite file `.env`.
- Osservazione e gestione visuale dei container tramite Docker Desktop.

11. Cosa dimostra il laboratorio

Il laboratorio dimostra come un'applicazione possa essere suddivisa in servizi indipendenti ma coordinati. L'applicazione web, il database e lo strumento di amministrazione non devono essere installati e configurati come un unico blocco sul sistema operativo: ciascun componente vive nel proprio container e Docker Compose descrive come avviarli e farli comunicare.

Questo approccio rende l'ambiente ripetibile, più facile da ricreare e più semplice da comprendere. Il valore principale del laboratorio è quindi aver visto in pratica il ciclo completo: immagine, container, rete, volume, variabili d'ambiente, servizi multipli e gestione tramite Docker Desktop.